

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
```

Double-click (or enter) to edit

```
df=pd.read_csv("customer_retail csv file.csv")
```

```
print(df.head())
```

	InvoiceNo	StockCode	...	CustomerID	Country
0	536365	85123A	...	17850.0	United Kingdom
1	536365	71053	...	17850.0	United Kingdom
2	536365	84406B	...	17850.0	United Kingdom
3	536365	84029G	...	17850.0	United Kingdom
4	536365	84029E	...	17850.0	United Kingdom

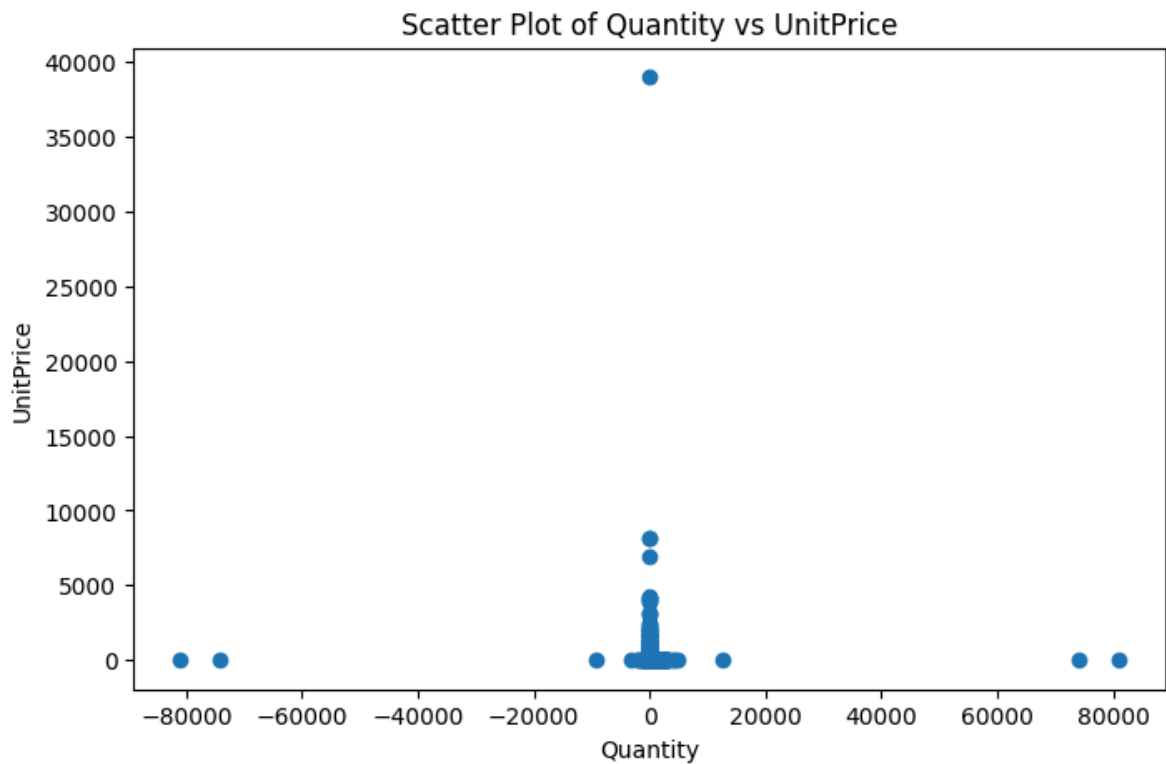
[5 rows x 8 columns]

```
df=df.dropna()
```

```
df=df[['Quantity','UnitPrice','Country']]
encoder=LabelEncoder()
```

```
df['Country_encoded']=encoder.fit_transform(df['Country'])
```

```
plt.figure(figsize=(8,5))
plt.scatter(df['Quantity'],df['UnitPrice'])
plt.xlabel('Quantity')
plt.ylabel('UnitPrice')
plt.title('Scatter Plot of Quantity vs UnitPrice')
plt.show()
```



```
x=df[['Quantity','UnitPrice']]
y=df['Country_encoded']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
print("LogisticRegression")
log_model=LogisticRegression(max_iter=1000)
log_model.fit(x_train,y_train)
y_pred=log_model.predict(x_test)
lr_accuracy=accuracy_score(y_test,y_pred)
print("Accuracy:",lr_accuracy)
cm=confusion_matrix(y_test,y_pred)
print("Confusion Matrix:")
print(cm)
```

```
LogisticRegression
Accuracy: 0.8898680038345255
Confusion Matrix:
[[ 0  0  0 ...  0 229  0]
 [ 0  0  0 ...  0  78  0]
 [ 0  0  0 ...  0   5  0]
 ...
 [ 0  0  0 ...  0  13  0]
 [ 0  0  0 ...  0 72405  0]
 [ 0  0  0 ...  0   43  0]]
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: Convergen
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
 n_iter_i = _check_optimize_result(

```
print("Decision Tree")
dt_model=DecisionTreeClassifier()
dt_model.fit(x_train,y_train)
y_pred=dt_model.predict(x_test)
dt_accuracy=accuracy_score(y_test,y_pred)
```

```
print("Accuracy:",dt_accuracy)
print(confusion_matrix(y_test,y_pred))
```

```
Decision Tree
Accuracy: 0.8903227392276872
[[ 13  0  0 ...  0 197  0]
 [  0  0  0 ...  0  75  0]
 [  0  0  0 ...  0   5  0]
 ...
 [  0  0  0 ...  0  13  0]
 [ 21  0  0 ... 172220  0]
 [  0  0  0 ...  0  43  0]]
```

```
print("KNN")
knn_model=KNeighborsClassifier()
knn_model.fit(x_train,y_train)
y_pred=knn_model.predict(x_test)
knn_accuracy=accuracy_score(y_test,y_pred)
print("Accuracy:",knn_accuracy)
print(confusion_matrix(y_test,y_pred))
```

```
KNN
Accuracy: 0.8842268269301674
[[ 25  0  0 ...  0 181  0]
 [  0  0  0 ...  0  75  0]
 [  0  0  0 ...  0   5  0]
 ...
 [  0  0  0 ...  0  12  0]
 [ 41  2  0 ...  071636  0]
 [  0  0  0 ...  0  43  0]]
```

```
models=[
    "Logistic Regression",
    "Decision Tree",
    "KNN"
]
accuracies=[
    lr_accuracy,
    dt_accuracy,
    knn_accuracy
]
plt.figure(figsize=(8,5))
plt.bar(models,accuracies)
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Model Accuracy Comparison")
plt.show()
```

